YOLOv8 Selective Class Detection GUI

1. Executive Summary

The objective of this project was to design and implement a Graphical User Interface (GUI) application in Python capable of performing **real-time object detection** using the **YOLOv8** model with a critical functional requirement: **selective class filtering**. The final application successfully provides a live webcam feed and allows users to explicitly choose which of the 80 COCO classes they wish to detect. This selective approach enhances performance and user focus, making the application ideal for targeted monitoring, such as quality control or specific surveillance tasks. The GUI was built using **Tkinter** and **OpenCV**, and includes robust controls for starting, pausing, and stopping the video stream.

2. Technical Architecture

The application relies on a combination of Python libraries, each fulfilling a specialized role in the detection and interface pipeline.

Component	Technology / Library	Role in Application
Model	**YOLOv8n/m/l** **(Nano)** (via ultralytics)	High-speed, state-of-the-art object detection engine used for inference. Other modes like (medium ,large) for better accuracy
GUI Framework	**Tkinter**	Provides the user interface (window, buttons, checkboxes) and manages the main application loop.
Video Processing	**OpenCV (**cv2**)**	Handles webcam initialization, frame capture, image manipulation, and drawing bounding boxes.
Image Handling	**PIL/Pillow (**ImageTk**)**	Converts OpenCV frames (NumPy arrays) into image formats compatible with Tkinter.
Control Logic	**Python (Class-Based)**	Manages the application state, filtering logic, and the critical video loop scheduling (tkinter.after).

3. Key Functional Features

The GUI was structured to provide granular control over both class selection and video stream management.

3.1 Selective Class Filtering

The primary feature allows users to filter detections in real time, optimizing the model's performance by only processing requested classes.

- **Categorized Toggles:** The 80 COCO classes are logically grouped (e.g., "Vehicles," "Kitchenware") and displayed via category buttons to improve navigation.
- **Real-Time Filtering:** The application gathers the IDs of all checked classes and passes this list directly to the YOLOv8 model's prediction function. This ensures the model **only detects the selected objects**.
- **Toggle Controls:** Dedicated buttons are provided for **"All: Toggle"** (global check/uncheck) and **"Toggle Current Group"** (check/uncheck all visible categories).

3.2 Live Video Stream Controls

Three specialized buttons provide essential control over the webcam without needing to shut down the application.

Control	Action	Mechanism
Pause	Halts the detection and video display loop, freezing the current frame and showing a "Paused" status. **The webcam remains open.**	Sets a boolean flag (self.is_running = False) and cancels the tkinter.after() scheduled loop.
Continue	Restarts the detection and video processing from the paused state.	Resets the flag (self.is_running = True) and immediately calls the video_loop function.
Stop	Cleanly shuts down the entire application.	Releases the cv2.VideoCapture object resource and terminates the Tkinter instance.

4. GUI Layout and Usability

The GUI design was finalized to place controls in the most intuitive locations for user access:

- **Top Bar:** Contains the **"All: Toggle"** button and the primary **Category Buttons** (e.g., "People & Animals," "Vehicles & Transport").
- **Control Bar (Below Categories):** This critical row contains the label for the currently displayed category, the **three dedicated video controls (Pause, Continue, Stop)**, and the **"Toggle Current Group"** button, placing all high-frequency controls together.
- **Checkbox Grid:** Displays the individual class checkboxes for the selected category in a tidy, multi-column grid.

- **Video Display Area:** The largest central area dedicated to displaying the **live, annotated video feed**.

5. Implementation of Selective Filtering

The effectiveness of the selective filtering lies in how the class list interacts with the YOLOv8 API:

1. **State Capture:** The application dynamically creates a list of integer IDs for every object class that has its corresponding **Tkinter BooleanVar** set to True (i.e., every checked box).
2. **API Integration:** This list of selected class IDs is passed as a parameter (classes=[...]) directly into the YOLO model's inference call:
3. # YOLO inference call only processes classes in target_class_ids
4. results = model(frame, classes=target_class_ids)
5. **Result:** By constraining the model at the inference stage, the application avoids the overhead of detecting and post-processing unwanted classes, resulting in a responsive and focused detection stream.

Python code:

```
import tkinter as tk
from tkinter import messagebox
from PIL import Image, ImageTk
import cv2
import numpy as np
from ultralytics import YOLO

# --- Configuration ---
MODEL_NAME = 'yolov8m.pt'
DISPLAY_WIDTH = 640
DISPLAY_HEIGHT = 480
WEBCAM_ID = 1
CONFIDENCE_THRESHOLD = 0.2
LIME_GREEN = (0, 255, 0) # BGR format: (Blue, Green, Red)

# All 80 COCO classes, split into 7 well-defined categories
CLASSIFIED_COCO_CLASSES = {
  "People & Animals": {
    0: 'person', 14: 'bird', 15: 'cat', 16: 'dog', 17: 'horse', 18: 'sheep',
    19: 'cow', 20: 'elephant', 21: 'bear', 22: 'zebra', 23: 'giraffe'
  },
  "Vehicles & Transport": {
    1: 'bicycle', 2: 'car', 3: 'motorcycle', 4: 'airplane', 5: 'bus',
    6: 'train', 7: 'truck', 8: 'boat'
  },
  "Traffic & Outdoors": {
```

```
        9: 'traffic light', 10: 'fire hydrant', 11: 'stop sign', 12: 'parking meter',
        13: 'bench'
    },
    "Sports & Accessories": {
        24: 'backpack', 25: 'umbrella', 26: 'handbag', 27: 'tie', 28: 'suitcase',
        29: 'frisbee', 30: 'skis', 31: 'snowboard', 32: 'sports ball', 33: 'kite',
        34: 'baseball bat', 35: 'baseball glove', 36: 'skateboard', 37: 'surfboard',
        38: 'tennis racket'
    },
    "Kitchenware & Food": {
        39: 'bottle', 40: 'wine glass', 41: 'cup', 42: 'fork', 43: 'knife',
        44: 'spoon', 45: 'bowl', 46: 'banana', 47: 'apple', 48: 'sandwich',
        49: 'orange', 50: 'broccoli', 51: 'carrot', 52: 'hot dog', 53: 'pizza',
        54: 'donut', 55: 'cake'
    },
    "Furniture & Household": {
        56: 'chair', 57: 'couch', 58: 'potted plant', 59: 'bed', 60: 'dining table',
        61: 'toilet', 73: 'book', 74: 'clock', 75: 'vase', 76: 'scissors',
        77: 'teddy bear', 78: 'hair drier', 79: 'toothbrush'
    },
    "Devices & Appliances": {
        62: 'tv', 63: 'laptop', 64: 'mouse', 65: 'remote', 66: 'keyboard',
        67: 'cell phone', 68: 'microwave', 69: 'oven', 70: 'toaster',
        71: 'sink', 72: 'refrigerator'
    }
}

# Flatten the categories for detection logic
SELECTABLE_CLASSES = {}
for group in CLASSIFIED_COCO_CLASSES.values():
    SELECTABLE_CLASSES.update(group)

# --- Application Class ---
class YoloWebcamFilterApp:
    def __init__(self, master):
        self.master = master
        master.title("YOLOv8 Webcam Class Filter")
        master.configure(bg="#f4f4f4")

        # Video State Control
        self.is_running = True
        self.video_loop_id = None # To store the ID of the recurring Tkinter job

        # Initialization
        self.model = self._load_model()
        self.video_capture = self._init_webcam()
        self.current_tk_image = None
```

```
        self.class_vars = {}
        self.checkbox_widgets = {}
        self.current_category_name = None
        self.class_colors = self._generate_colors()

        self._setup_layout()

        # Initialize the display with the first category
        first_category = next(iter(CLASSIFIED_COCO_CLASSES.keys()))
        self.show_category_classes(first_category)

        self.video_loop()

    def _load_model(self):
        try:
            model = YOLO(MODEL_NAME)
            print(f"YOLOv8 model ({MODEL_NAME}) loaded successfully.")
            return model
        except Exception as e:
            messagebox.showerror("Error", f"Failed to load YOLO model: {e}")
            self.master.destroy()
            return None

    def _init_webcam(self):
        cap = cv2.VideoCapture(WEBCAM_ID)
        if not cap.isOpened():
            messagebox.showerror("Error", "Could not open webcam. Check WEBCAM_ID or
permissions.")
            self.master.destroy()
            return None
        cap.set(cv2.CAP_PROP_FRAME_WIDTH, DISPLAY_WIDTH)
        cap.set(cv2.CAP_PROP_FRAME_HEIGHT, DISPLAY_HEIGHT)
        print(f"Webcam {WEBCAM_ID} initialized.")
        return cap

    def _generate_colors(self):
        colors = {}
        for class_id in SELECTABLE_CLASSES.keys():
            colors[class_id] = LIME_GREEN
        return colors

    def toggle_current_group(self):
        """Toggles all checkboxes in the CURRENTLY DISPLAYED category."""
        if not self.current_category_name:
            return

        class_dict = CLASSIFIED_COCO_CLASSES.get(self.current_category_name)
        if not class_dict:
```

```
            return

        # Determine if ALL currently displayed classes are checked
        all_checked = all(self.class_vars.get(class_id, tk.BooleanVar(value=False)).get() for class_id in
class_dict.keys())

        # If all are checked, the new state is False (uncheck). Otherwise, the new state is True (check).
        new_state = not all_checked

        for class_id in class_dict.keys():
            if class_id in self.class_vars:
                self.class_vars[class_id].set(new_state)

    def toggle_all_classes(self):
        """Checks or unchecks ALL class variables in the application (hidden or visible)."""

        all_are_checked = all(var.get() for var in self.class_vars.values())
        new_state = not all_are_checked

        for var in self.class_vars.values():
            var.set(new_state)

    def show_category_classes(self, category_name):
        """Hides all checkboxes and shows only those belonging to the selected category."""

        # 1. Hide ALL checkboxes
        for widget in self.checkbox_widgets.values():
            widget.grid_forget()

        class_dict = CLASSIFIED_COCO_CLASSES.get(category_name, {})

        # 2. Show only the relevant checkboxes, skipping row 0 for the label and controls
        cols = 8 # Number of columns for the sub-classes display
        for i, (class_id, class_name) in enumerate(class_dict.items()):
            row = (i // cols) + 1  # Start from row 1
            col = i % cols

            widget = self.checkbox_widgets.get(class_id)
            if widget:
                # Show the widget using grid()
                widget.grid(row=row, column=col, padx=5, pady=2, sticky="w")

        # 3. Update state tracking
        self.current_category_name = category_name
        self.class_label.config(text=f"Individual Toggles: {category_name}")

    def _setup_layout(self):
        """Sets up the GUI components with the webcam controls next to Toggle Current Group."""
```

```
        main_frame = tk.Frame(self.master, padx=10, pady=10, bg="#f4f4f4")
        main_frame.pack(fill=tk.BOTH, expand=True)

        # 1. Control Panel
        control_frame = tk.LabelFrame(main_frame, text="YOLOv8 Class Filtering", padx=10, pady=10,
bg="white", font=('Inter', 10, 'bold'))
        control_frame.pack(fill=tk.X, pady=5)

        # --- Pre-Setup (Individual Checkboxes Frame) ---
        self.class_frame = tk.Frame(control_frame, bg="white")
        self._pre_generate_all_checkboxes()
        # -----------------------------------

        # A. Top Bar: All Toggle, and Category Buttons (Packed LEFT to RIGHT)
        category_btn_frame = tk.Frame(control_frame, bg="white")
        category_btn_frame.pack(fill=tk.X, pady=(0, 10))

        # All Toggle button (Leftmost in the top bar)
        tk.Button(category_btn_frame, text="All: Toggle", command=self.toggle_all_classes,
                  bg="#f0f0f0", relief=tk.RAISED, font=('Inter', 9, 'bold'),
                  fg="#444444", activebackground="#dddddd").pack(side=tk.LEFT, padx=(0, 10), pady=5)

        # Category Buttons
        for category_name in CLASSIFIED_COCO_CLASSES.keys():
            tk.Button(
                category_btn_frame,
                text=category_name,
                command=lambda name=category_name: self.show_category_classes(name),
                bg="#e8f0fe", relief=tk.RAISED, font=('Inter', 9),
                activebackground="#c0d8ff"
            ).pack(side=tk.LEFT, padx=5, pady=5)

        # B. Second Bar (Row 0 of the class_frame grid): Label, Pause/Continue/Stop, and Toggle
Current Group

        # 1. Row 0, Column 0: Label
        self.class_label = tk.Label(self.class_frame, text="Individual Toggles: Select a Category",
bg="white", font=('Inter', 9))
        self.class_label.grid(row=0, column=0, columnspan=2, sticky="w", padx=0, pady=5)

        # 2. Row 0, Columns 5-7: Pause, Continue, Stop buttons (Aligned right)
        # We use a separate frame for these buttons to group them and align them easily
        control_btn_frame = tk.Frame(self.class_frame, bg="white")
        control_btn_frame.grid(row=0, column=3, columnspan=4, sticky='e', padx=10, pady=5)

        tk.Button(control_btn_frame, text="⏸️ Pause", command=self._pause_webcam,
                  bg="#ffc0c0", relief=tk.RAISED, font=('Inter', 9, 'bold'),
```

```
            fg="#444444").pack(side=tk.LEFT, padx=2)

    tk.Button(control_btn_frame, text="▶ Continue", command=self._continue_webcam,
            bg="#c0ffc0", relief=tk.RAISED, font=('Inter', 9, 'bold'),
            fg="#444444").pack(side=tk.LEFT, padx=2)

    # The Stop button calls the on_closing method which performs cleanup
    tk.Button(control_btn_frame, text="■ Stop", command=self.on_closing,
            bg="#c0c0ff", relief=tk.RAISED, font=('Inter', 9, 'bold'),
            fg="#444444").pack(side=tk.LEFT, padx=2)

    # 3. Row 0, Column 7: Toggle Current Group Button (Placed rightmost)
    tk.Button(
       self.class_frame, text="Toggle Current Group",
       command=self.toggle_current_group,
       bg="#ffe0e0", relief=tk.RAISED, font=('Inter', 8),
       activebackground="#ffc0c0", fg="#444444"
    ).grid(row=0, column=7, sticky="e", padx=(10, 0), pady=5)


    # --- PACKING ORDER ---

    # 1. Pack the frame containing the individual checkboxes, label, and controls
    self.class_frame.pack(fill=tk.X)

    # ---------------------------------

    # 2. Image Display Area
    self.image_label = tk.Label(main_frame, text="Initializing Webcam...", bg="#cccccc",
width=DISPLAY_WIDTH // 8, height=DISPLAY_HEIGHT // 8)
    self.image_label.pack(fill=tk.BOTH, expand=True, pady=10)

  def _pre_generate_all_checkboxes(self):
    """Generates all 80 BooleanVars and Checkbutton widgets once at startup."""

    all_classes_list = list(SELECTABLE_CLASSES.items())

    for class_id, class_name in all_classes_list:
       # 1. Create and store the BooleanVar
       var = tk.BooleanVar(value=False)
       self.class_vars[class_id] = var

       # 2. Create the Checkbutton widget, using the now-defined self.class_frame
       chk = tk.Checkbutton(
          self.class_frame,
          text=class_name,
          variable=var,
          bg="white",
```

```
            font=('Inter', 9),
            anchor="w"
        )
        # 3. Store the widget and initially hide it
        self.checkbox_widgets[class_id] = chk
        chk.grid_forget()

    def _pause_webcam(self):
        """Pauses the video processing loop, keeps webcam open, and shows 'Paused' message."""
        self.is_running = False
        if self.video_loop_id:
            # Cancel any scheduled video_loop call
            self.master.after_cancel(self.video_loop_id)

        # Use the existing image_label to display the text

        # Clear the image reference to ensure text is visible
        self.image_label.image = None

    def _continue_webcam(self):
        """Resumes the video processing loop."""
        if not self.is_running:
            self.is_running = True
            # Clear the paused message and restart the loop immediately
            self.image_label.configure(text="Running...", fg="black", font=('Inter', 9))
            self.video_loop()

    def display_frame(self, cv_image):
        rgb_image = cv2.cvtColor(cv_image, cv2.COLOR_BGR2RGB)
        pil_image = Image.fromarray(rgb_image)
        self.current_tk_image = ImageTk.PhotoImage(pil_image)
        # Reset compound to display image and clear the paused state font/color
        self.image_label.configure(image=self.current_tk_image, text="", compound="none",
fg="black", font=('Inter', 9))
        self.image_label.image = self.current_tk_image

    def detect_and_filter(self, frame):
        target_class_ids = [
            class_id
            for class_id, var in self.class_vars.items()
            if var.get()
        ]

        if not target_class_ids:
            return frame

        annotated_frame = frame.copy()
```

```
    # YOLOv8 optimization: only detect the classes we care about
    # Note: Set verbose=False to keep the console clean during the loop
    results = self.model(annotated_frame, verbose=False, conf=CONFIDENCE_THRESHOLD,
classes=target_class_ids)

    for result in results:
        boxes = result.boxes
        for box in boxes:
            class_id = int(box.cls.cpu().numpy()[0])

            x1, y1, x2, y2 = [int(x) for x in box.xyxy[0].cpu().numpy()]
            confidence = box.conf.cpu().numpy()[0]

            class_name = self.model.names[class_id]
            color = self.class_colors[class_id]

            cv2.rectangle(annotated_frame, (x1, y1), (x2, y2), color, 2)
            label = f"{class_name} {confidence:.2f}"
            cv2.putText(annotated_frame, label, (x1, y1 - 10),
                        cv2.FONT_HERSHEY_SIMPLEX, 0.7, color, 2)

    return annotated_frame

  def video_loop(self):
    # Only run if the state is set to running
    if not self.is_running:
        return

    if self.video_capture is None or not self.video_capture.isOpened():
        # If the webcam is somehow closed while running, stop the loop.
        self.is_running = False
        return

    ret, frame = self.video_capture.read()

    if ret:
        enhanced_frame = cv2.medianBlur(frame, 3)
        annotated_frame = self.detect_and_filter(frame)
        self.display_frame(annotated_frame)
    else:
        # Handle case where frame could not be read (e.g., camera disconnected)
        print("Error: Could not read frame from camera.")
        self.is_running = False

    # Store the job ID and schedule the next iteration

    # If running, schedule the next frame update
    if self.is_running:
```

```
            self.video_loop_id = self.master.after(10, self.video_loop)

    def on_closing(self):
        """Stops the loop, releases the webcam, and quits the application."""
        if messagebox.askokcancel("Quit", "Do you want to quit and stop the webcam?"):
            # Set running to false
            self.is_running = False

            if self.video_loop_id:
                # Cancel any pending 'after' calls
                self.master.after_cancel(self.video_loop_id)

            if self.video_capture and self.video_capture.isOpened():
                # Release the webcam resource
                self.video_capture.release()

            self.master.destroy()

# --- Main Execution ---
if __name__ == "__main__":

    root = tk.Tk()
    app = YoloWebcamFilterApp(root)
    # Ensure the window close button calls the clean-up method
    root.protocol("WM_DELETE_WINDOW", app.on_closing)
    root.mainloop()
```